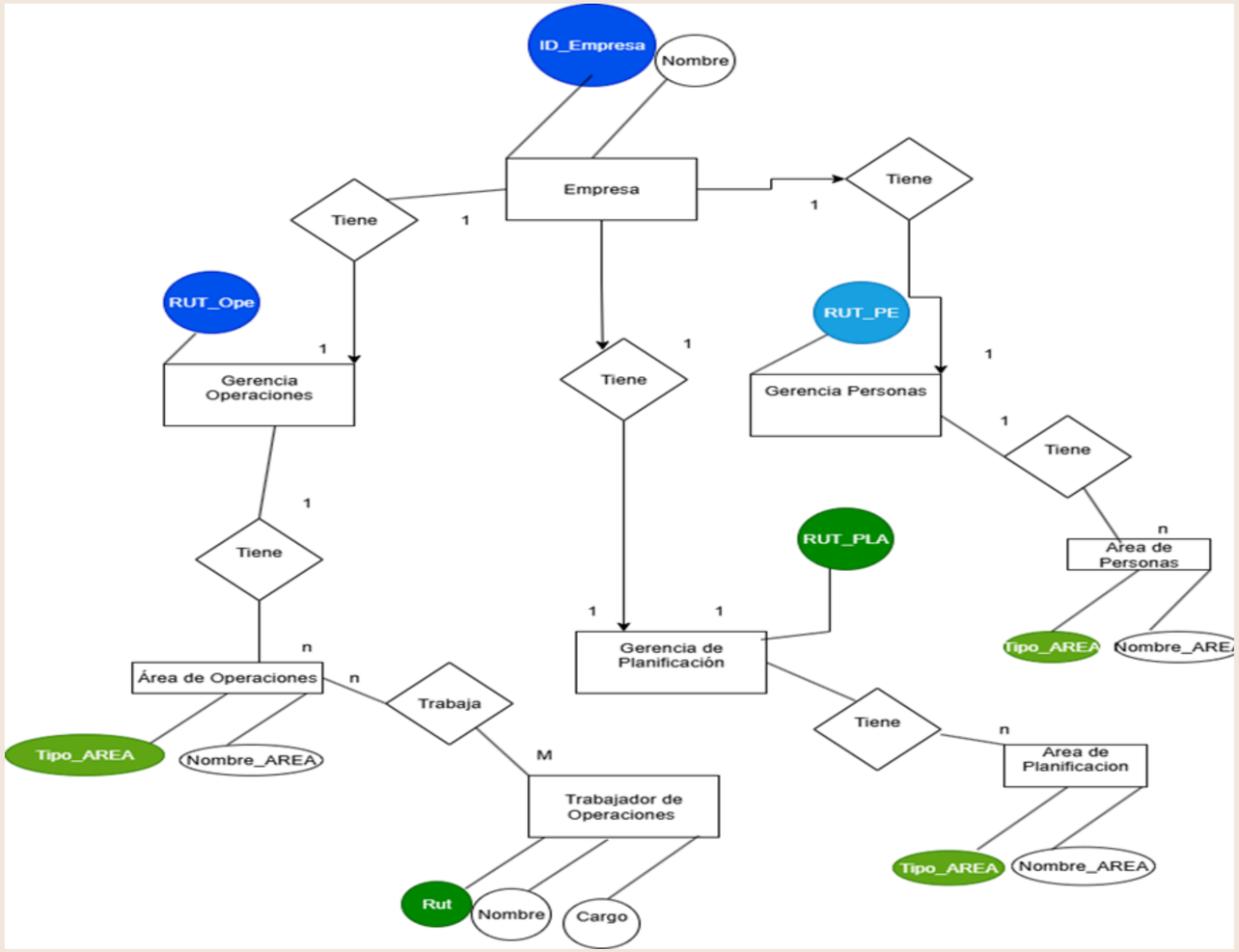
Presentación Proyecto de Bases de Datos

Integrantes

Pedro Rojas
Dylan Barahona
Joaquín Abarzúa
Alejandro Arias
Cristian San Martin

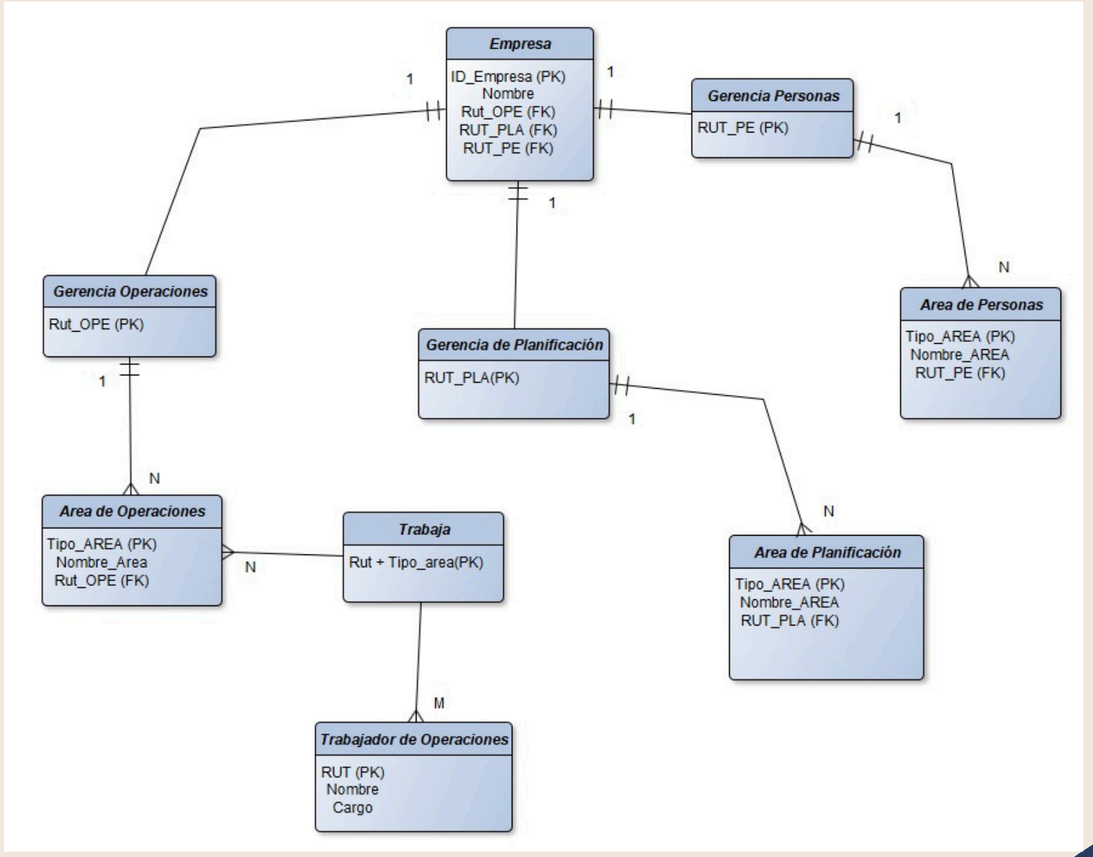
Modelo E-R y Modelo Relacional

Lo primero es recordar los modelos E-R y Relacional construidos por el equipo, primero se presentará el modelo E-R el cual después de recibir feedback por parte del cliente, este se encuentra correcto:



Modelo Relacional

Para el modelo relacional, después de recibir feedback por parte del cliente, se realizaron algunas modificaciones, lo primero de todo es que se eliminaron las claves foraneas de la tabla Trabaja, y se agrego una clave foranea en la tabla de area de operaciones por la relacion 1:N



Normalización

1FN – Primera forma normal

Cada celda contiene un valor atomico (es decir es indivisible), no pueden ser listas, contenedores, arrays, etc
En caso no se cumpla 1FN, se agrega una fila según corresponda, por ejemplo:
Pedro, Jorge 25,26 → arreglado
Pedro 25
Jorge 26

2FN – Segunda Forma Normal

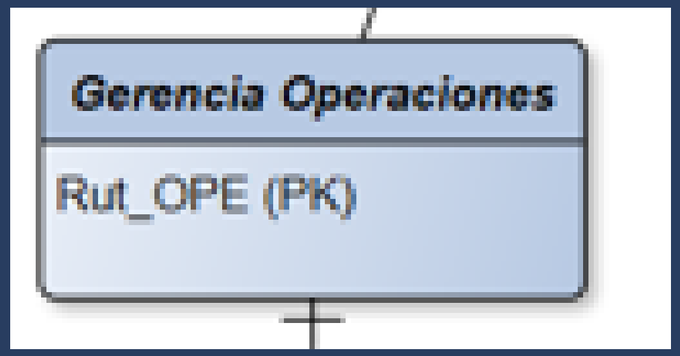
Se debe estar en 1FN
Cada atributo depende completamente de la clave primaria, si existe dependencia funcional se rompe 2FN: “Decimos que un atributo depende funcionalmente de otro si su valor está determinado por el otro”
La solución es separar en tablas

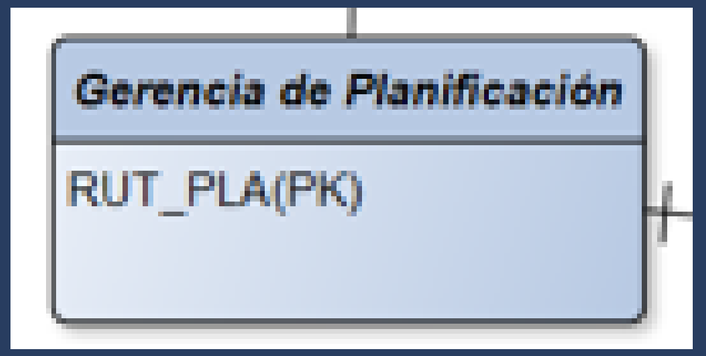
3FN – Tercera Forma Normal

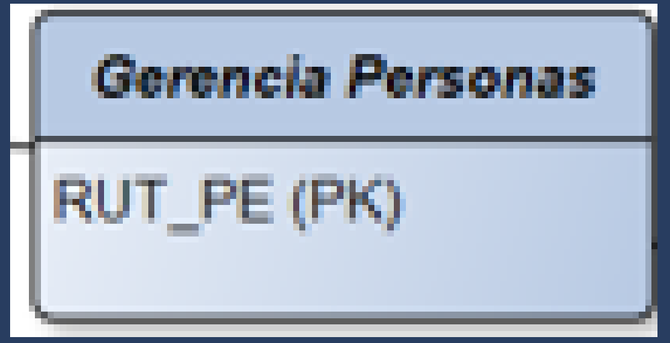
Se debe estar en 2FN
No deben existir dependencias transitivas: Una dependencia transitiva ocurre cuando un atributo no clave depende de otro atributo que tampoco es clave, pero a través de la clave primaria.
La solución es la eliminación de atributos dependientes y se crean nuevas tablas para ellos.

Normalización de las tablas del modelo relacional

Primero se realizará el analisis con las gerencias, ya que no son muy dificiles de transformar a 3FN

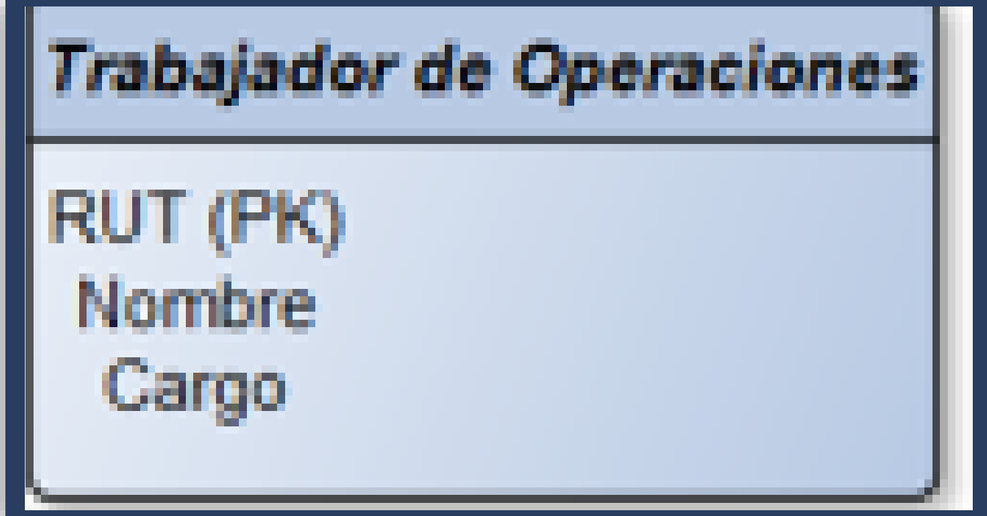


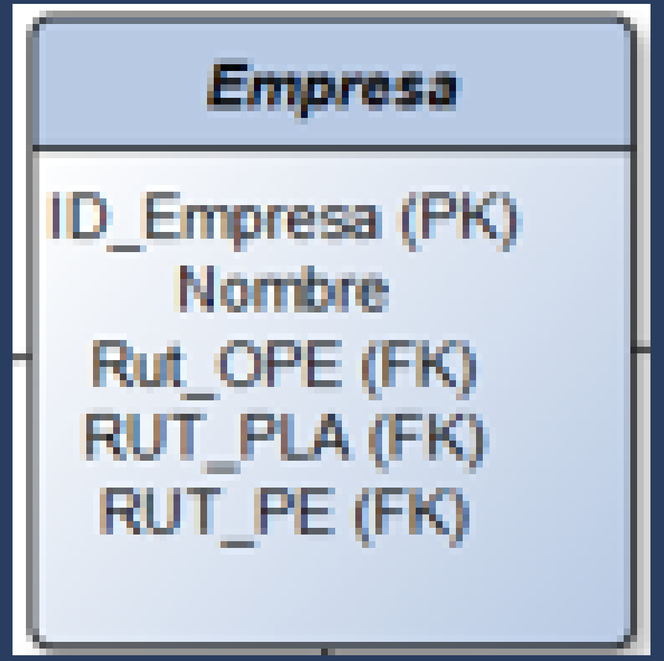




Ahora se realizará el analisis a Empresa, Trabaja y Trabajador de Operaciones





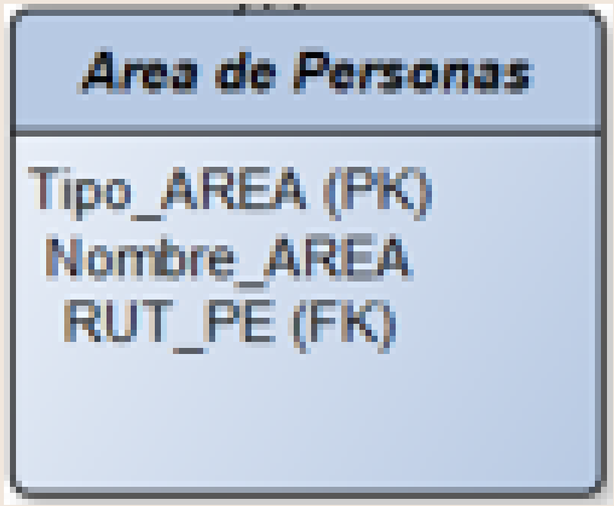


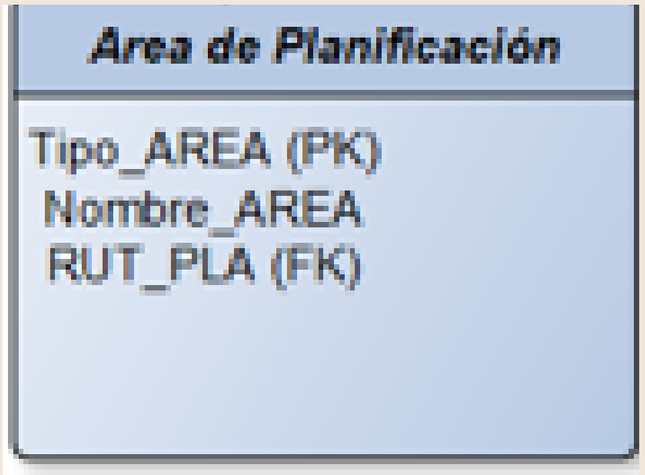
Normalización de las tablas del modelo relacional 2

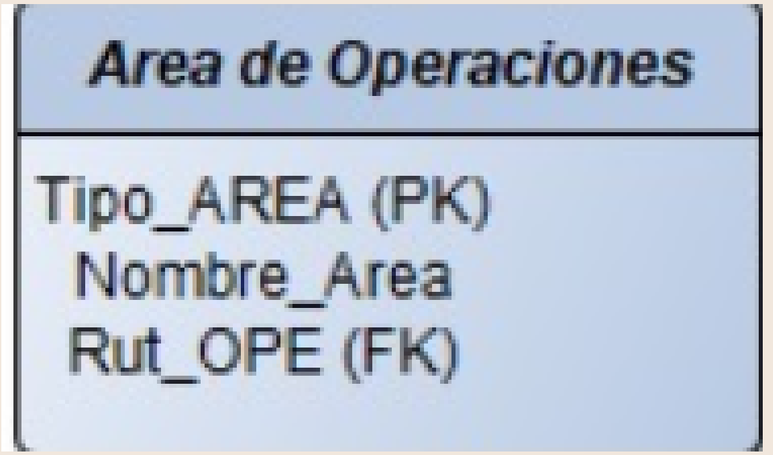
A continuación se realizará el analisis de las tablas faltantes las cuales son las áreas de operaciones, persona y planificacion

Problemática con el área de operaciones en general, estas cumplen 1FN, ya que se tienen datos atomicos, es decir no se tienen listas, ni arrays, se tienen atributos únicos con su respectiva fila y columna. Sin embargo, el problema es que el atributo nombre_area no depende en su totalidad de la clave primaria tipo_area, solamente de manera parcial, esto se debe a que según el tipo de área (superintendencia, intendencia entre otros) sera el nombre del área por lo cual se rompe 2FN, esto ocurre para las 3 áreas del modelo relacional

Para solucionar esto se debe separar el tipo_area y el nombre en una tabla aparte para que no exista dependencia parcial en la tabla de operaciones, se tendría lo siguiente:

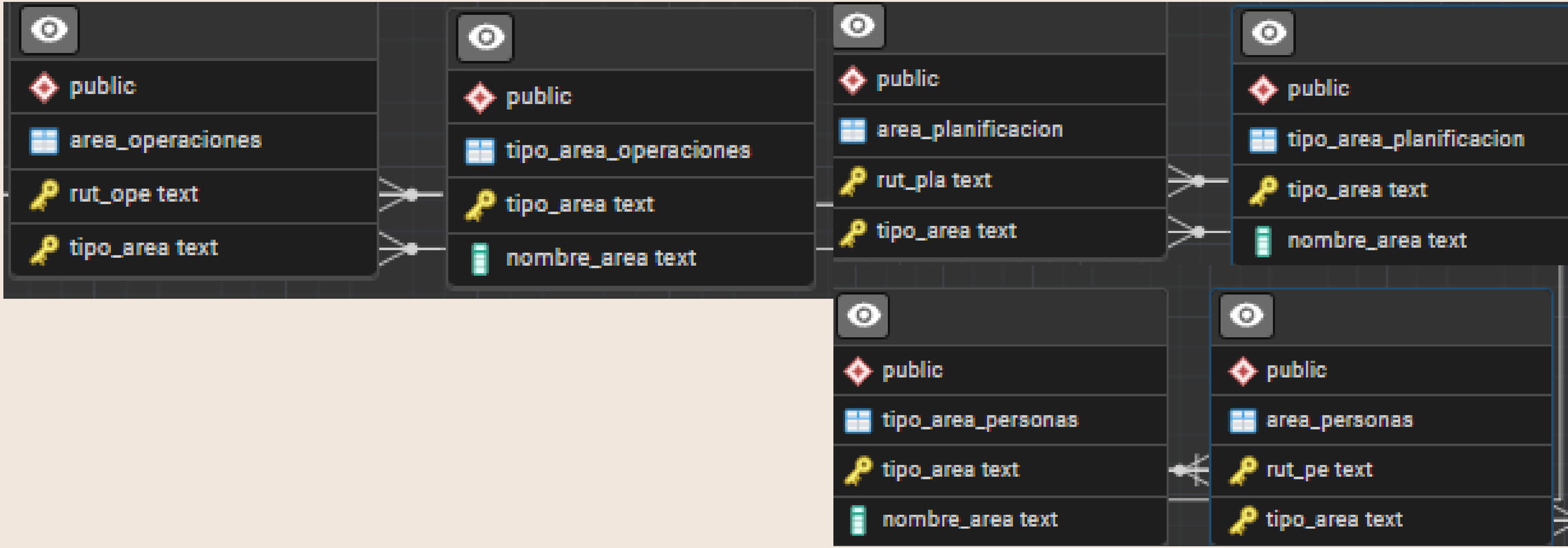






Normalizacion de las tablas de área

Para poder normalizar las tablas de áreas, entonces es necesario crear una nueva tabla para cada area, el cual se llamara tipo área y tendrá la dependencia parcial que existía entre tipo área y nombre área, esta tabla se conectara con las áreas originales, en donde solo se conservara la PK del Rut, y las foráneas respectivas, además se agregaran las foráneas de las nuevas tablas de tipo área



Modelo Relacional - aSQL

Ya teniendo todas las tablas en 3FN, se debe pasar del modelo relacional normalizado a tablas en PostgreSQL, en este caso se utilizó PgAdmin4 para la realización de este proceso y a continuación se presentan los bloques de códigos

```sql
CREATE TABLE Tipo_Area_Operaciones (
  TIPO_AREA TEXT PRIMARY KEY,
  NOMBRE_AREA TEXT NOT NULL
);

CREATE TABLE Area_Operaciones (
  RUT_OPE TEXT NOT NULL,
  TIPO_AREA TEXT NOT NULL,
  PRIMARY KEY (RUT_OPE, TIPO_AREA),
  FOREIGN KEY (RUT_OPE) REFERENCES Gerencia_Operaciones(RUT_OPE),
  FOREIGN KEY (TIPO_AREA) REFERENCES Tipo_Area_Operaciones(TIPO_AREA)
);


CREATE TABLE Tipo_Area_Planificacion (
  TIPO_AREA TEXT PRIMARY KEY,
  NOMBRE_AREA TEXT NOT NULL
);

CREATE TABLE Area_Planificacion (
  RUT_PLA TEXT NOT NULL,
  TIPO_AREA TEXT NOT NULL,
  PRIMARY KEY (RUT_PLA, TIPO_AREA),
  FOREIGN KEY (RUT_PLA) REFERENCES Gerencia_Planificacion(RUT_PLA),
  FOREIGN KEY (TIPO_AREA) REFERENCES Tipo_Area_Planificacion(TIPO_AREA)
);


CREATE TABLE Tipo_Area_Personas (
  TIPO_AREA TEXT PRIMARY KEY,
  NOMBRE_AREA TEXT NOT NULL
);

CREATE TABLE Area_Personas (
  RUT_PE TEXT NOT NULL,
  TIPO_AREA TEXT NOT NULL,
  PRIMARY KEY (RUT_PE, TIPO_AREA),
  FOREIGN KEY (RUT_PE) REFERENCES Gerencia_Personas(RUT_PE),
  FOREIGN KEY (TIPO_AREA) REFERENCES Tipo_Area_Personas(TIPO_AREA)
);
```

```sql
CREATE TABLE Empresa (
  ID_EMPRESA SERIAL PRIMARY KEY,
  NOMBRE TEXT NOT NULL,
  RUT_OPE TEXT UNIQUE,
  RUT_PLA TEXT UNIQUE,
  RUT_PE  TEXT UNIQUE
);

CREATE TABLE Gerencia_Operaciones (
  RUT_OPE TEXT PRIMARY KEY,
  FOREIGN KEY (RUT_OPE) REFERENCES Empresa(RUT_OPE)
);

CREATE TABLE Gerencia_Planificacion (
  RUT_PLA TEXT PRIMARY KEY,
  FOREIGN KEY (RUT_PLA) REFERENCES Empresa(RUT_PLA)
);

CREATE TABLE Gerencia_Personas (
  RUT_PE TEXT PRIMARY KEY,
  FOREIGN KEY (RUT_PE) REFERENCES Empresa(RUT_PE)
);

CREATE TABLE Trabajador_Operaciones (
  RUT TEXT PRIMARY KEY,
  NOMBRE TEXT NOT NULL,
  CARGO TEXT NOT NULL
);

CREATE TABLE Trabaja (
  RUT TEXT NOT NULL,
  RUT_OPE TEXT NOT NULL,    -
  TIPO_AREA TEXT NOT NULL,
  PRIMARY KEY (RUT, RUT_OPE, TIPO_AREA),
  FOREIGN KEY (RUT) REFERENCES Trabajador_Operaciones(RUT),
  FOREIGN KEY (RUT_OPE, TIPO_AREA)
    REFERENCES Area_Operaciones(RUT_OPE, TIPO_AREA)
);
```

Poblamiento de la Base de Datos

Luego de la construcción de las tablas en PostgreSQL, es necesario poblar la base de datos, para realizar pruebas, es por ello que se tiene el siguiente código para poblar la base de datos, todo esto se baso en la lógica de negocios y en la industria chilena

```
NSERT INTO Empresa (nombre, rut_ope, rut_pla, rut_pe)
ALUES (
  'CIQ Minería S.A.',
  '77.123.456-1',    -- Gerencia de Operaciones
  '77.123.457-0',    -- Gerencia de Planificación
  '77.123.458-9'     -- Gerencia de Personas
;

NSERT INTO Gerencia_Operaciones (rut_ope) VALUES ('77.123.456-1');
NSERT INTO Gerencia_Planificacion (rut_pla) VALUES ('77.123.457-0');
NSERT INTO Gerencia_Personas (rut_pe) VALUES ('77.123.458-9');


NSERT INTO Tipo_Area_Operaciones (tipo_area, nombre_area) VALUES
'A01', 'Superintendencia de Tronadura'),
'A02', 'Superintendencia de Cargulo y Transporte'),
'A03', 'Superintendencia de Chancado y Correas');

NSERT INTO Area_Operaciones (rut_ope, tipo_area) VALUES
'77.123.456-1', 'A01'),
'77.123.456-1', 'A02'),
'77.123.456-1', 'A03');


NSERT INTO Tipo_Area_Planificacion (tipo_area, nombre_area) VALUES
'P01', 'Departamento de Programación Minera'),
'P02', 'Departamento de Control de Producción'),
'P03', 'Departamento de Logística y Transporte');

NSERT INTO Area_Planificacion (rut_pla, tipo_area) VALUES
'77.123.457-0', 'P01'),
'77.123.457-0', 'P02'),
'77.123.457-0', 'P03');

NSERT INTO Tipo_Area_Personas (tipo_area, nombre_area) VALUES
'R01', 'Área de Reclutamiento y Selección'),
'R02', 'Área de Capacitación y Desarrollo'),
'R03', 'Área de Relaciones Laborales');

NSERT INTO Area_Personas (rut_pe, tipo_area) VALUES
'77.123.458-9', 'R01'),
'77.123.458-9', 'R02'),
'77.123.458-9', 'R03');
```

```
INSERT INTO Trabajador_Operaciones (rut, nombre, cargo) VALUES
('18.456.789-5', 'Diego Morales', 'Supervisor de Tronadura'),
('19.234.567-4', 'Camila Rojas', 'Ingeniera de Transporte'),
('17.345.678-2', 'José Fuentes', 'Operador de Correas'),
('16.789.345-9', 'Daniel Contreras', 'Técnico Mecánico'),
('15.876.234-6', 'Rocío Herrera', 'Operadora de Planta'),
('21.654.987-5', 'Rodrigo Muñoz', 'Analista de Operaciones');


INSERT INTO Trabaja (rut, rut_ope, tipo_area) VALUES
('18.456.789-5', '77.123.456-1', 'A01'),
('19.234.567-4', '77.123.456-1', 'A02'),
('17.345.678-2', '77.123.456-1', 'A03'),
('16.789.345-9', '77.123.456-1', 'A03'),
('15.876.234-6', '77.123.456-1', 'A02'),
('21.654.987-5', '77.123.456-1', 'A01');
```

Consultas SQL – Pruebas a la Base de Datos

A continuación se presentan algunas consultas SQL que se le hicieron a la base de datos, luego del poblamiento para su testeo

1) Obtener los nombres de los trabajadores que pertenecen a la “Superintendencia de Carguío y Transporte”

```sql
SELECT t.nombre, t.cargo
FROM Trabaja tr
JOIN Trabajador_Operaciones t ON tr.rut = t.rut
JOIN Tipo_Area_Operaciones ta ON tr.tipo_area = ta.tipo_area
WHERE ta.nombre_area = 'Superintendencia de Carguío y Transporte';
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	nombre text	cargo text
1	Camila Rojas	Ingeniera de Transpo...
2	Rocío Herre...	Operadora de Planta

2) Contar cuantos trabajadores hay por área

```sql
SELECT ta.nombre_area, COUNT(*) AS cantidad_trabajadores
FROM Trabaja tr
JOIN Tipo_Area_Operaciones ta ON tr.tipo_area = ta.tipo_area
GROUP BY ta.nombre_area
ORDER BY cantidad_trabajadores DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	nombre_area text	cantidad_trabajadores bigint
1	Superintendencia de Chancado y Corre...	2
2	Superintendencia de Carguío y Transpo...	2
3	Superintendencia de Tronadura	2

SP y Triggers

Para la Base de Datos se nos exigia minimo 1 trigger y 1 stored procedure, es por ello que se construyeron 1 de cada uno siguiendo una lógica de negocios y considerando la relevancia que estos podrian tener para futuras actualizaciones de la Base de Datos

SP:Agregar un nuevo trabajador y se asigne a un área dentro de la Base de Datos

```sql
CREATE OR REPLACE PROCEDURE registrar_trabajador_operaciones(
    p_rut TEXT,
    p_nombre TEXT,
    p_cargo TEXT,
    p_tipo_area TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
       IF NOT EXISTS (SELECT 1 FROM Area_Operaciones WHERE tipo_area = p_tipo_area) THEN
        RAISE NOTICE 'El área % no existe en la Gerencia de Operaciones.', p_tipo_area;
        RETURN;
    END IF;


    INSERT INTO Trabajador_Operaciones (rut, nombre, cargo)
    VALUES (p_rut, p_nombre, p_cargo);

        INSERT INTO Trabaja (rut, rut_ope, tipo_area)
    VALUES (p_rut, '77.123.456-1', p_tipo_area);

    RAISE NOTICE 'Trabajador % registrado y asignado al área % correctamente.', p_nombre, p_tipo_area;
END;
$$;
```

```sql
CALL registrar_trabajador_operaciones(
  '22.345.678-1',
  'Gonzalo Vargas',
  'Operador de Chancado',
  'A03'
);
```

Data Output | Messages | Notifications

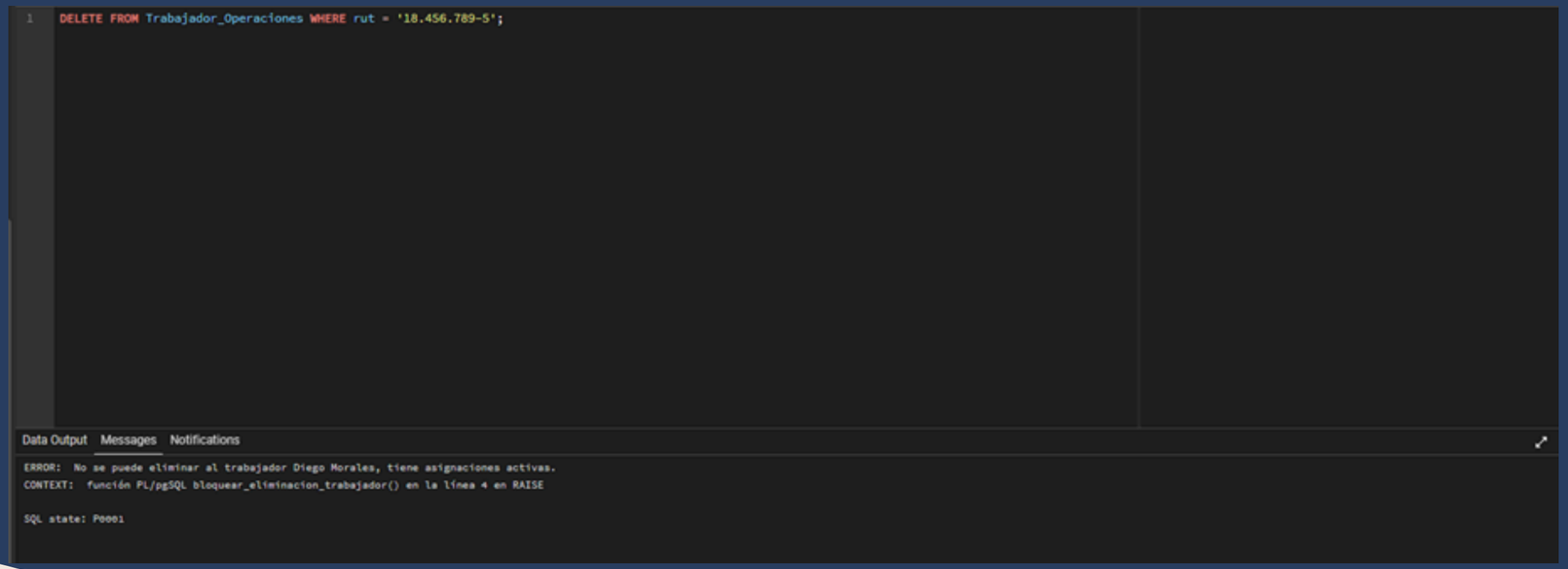
	rut [PK] text	nombre text	cargo text
1	18.456.789...	Diego Morales	Supervisor de Tronad...
2	19.234.567...	Camila Rojas	Ingeniera de Transpor...
3	17.345.678...	José Fuentes	Operador de Correas
4	16.789.345...	Daniel Contrer...	Técnico Mecánico
5	15.876.234...	Rocío Herrera	Operadora de Planta
6	21.654.987...	Rodrigo Muñoz	Analista de Operacion...
7	22.345.678...	Gonzalo Vargas	Operador de Chancado

Trigger

Se tiene el siguiente trigger: No se puedan borrar trabajadores los cuales se encuentran en sus respectivos cargos y activos

```sql
CREATE OR REPLACE FUNCTION bloquear_eliminacion_trabajador()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM Trabaja WHERE rut = OLD.rut) THEN
        RAISE EXCEPTION 'No se puede eliminar al trabajador %, tiene asignaciones activas.', OLD.nombre;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bloquear_eliminacion_trabajador
BEFORE DELETE ON Trabajador_Operaciones
FOR EACH ROW
EXECUTE FUNCTION bloquear_eliminacion_trabajador();
```

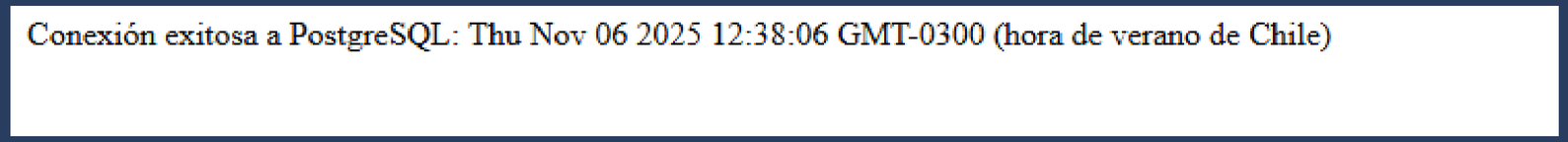


Aplicación WEB

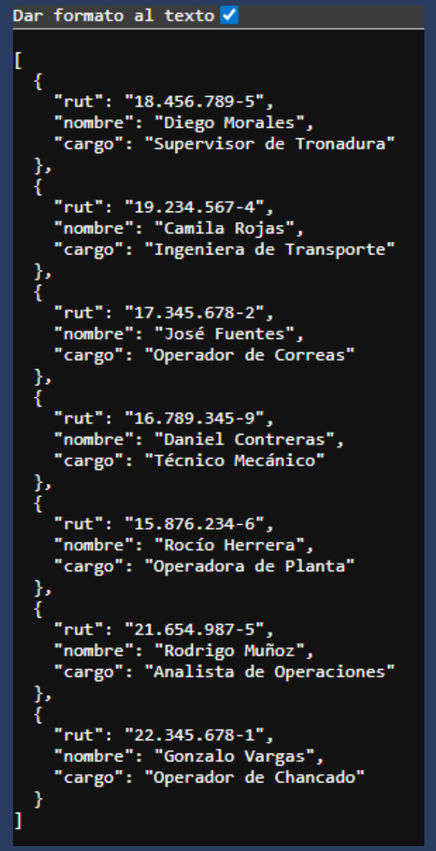
Para la presenta entrega se solicitaba crear una página utilizando el protocolo HTTP para poder mostrar datos de la Base de Datos, para realizar esto fue necesario utilizar Express, HTML y JS, además de eso se utilizo node.js el cual cuenta con muchisimas librerias de uso gratuito, en este caso pg el cual es una libreria de PostgreSQL en Node.JS



La página es sencilla, pero funcional contiene distintos enlaces en los cuales uno se puede ir desplazando para ver la conexión con la base de datos, por ejemplo:



Luego de observar la correcta conexión con PostgreSQL se pueden consultar los datos a traves de los enlaces, por ejemplo ver a los trabajadores



Conclusión

El desarrollo de la base de datos de CIQ , representó un proceso integral que abarcó todas las etapas fundamentales de la ingeniería de bases de datos: desde el análisis conceptual de los requerimientos de una organización real, hasta la implementación técnica, la automatización de procesos mediante lógica de negocio, y la integración con una aplicación web a través del protocolo HTTP.

El trabajo permitió no solo aplicar los principios teóricos de diseño relacional, sino también consolidar una visión práctica y funcional de cómo una base de datos correctamente estructurada puede convertirse en el eje de un sistema de información empresarial, además de eso se tuvieron beneficios tales como presentar una Base de Datos la cual podria funcionar en un entorno empresarial, y podria escalar a través de funciones como los SP y Triggers, también se corrigieron errores de la entrega 1 lo cual reforzó el procedimiento completo de la realizacion de una Base de Datos de entorno empresarial para el futuro como Ingenieros Civiles en Informática y Telecomunicaciones.

Por último se destacan algunas dificultades en entender conceptos como la normalizacion, afortunadamente el modelo relacional contenia bastantes tablas las cuales ya estaban en 3FN, por lo cual se pudo solucionar de una manera rápida y eficaz.

En síntesis el proyecto de Bases de Datos sirvió para consolidar conocimientos acerca del desarrollo y lógica de negocio utilizados en las empresas a la hora de la construcción de Bases de Datos las cuales deben satisfacer un problema.